A word about privileges in hardware

Privileges are levels of protection built into Microprocessors which allows or disallows a piece of software to send signals to the hardware. For example, a Bluetooth device driver must be able to send electric signals to the radio circuits to start emitting wireless signals. However, the UI component, even when presenting the options for pairing new devices should not be allowed to directly send signals to the hardware because:

a) The UI component might not know the low-level hardware details for emitting signals responsible for pairing a device.
b) The request might need to be authenticated first (e.g. the user must enter a password for connecting to a new device) and thus needs other parts of the OS to come into play.

In the second case, the UI components might need to be stopped or suspended for the time and the OS must first authenticate the user. The UI component might crash if it encountered an exception when trying to initiate. For this use case (and plenty others), the code which can control hardware is usually run in 'kernel space' or 'privileged mode'. Such code communicates with higher-level software components using a predefined and well-documented API presented by the OS on behalf of the driver.

There are multiple levels of privileges built into microprocessors. The more advanced and generic the microprocessor, more the number of privilege levels. If you really want to understand how OS interacts with hardware, you can go through the Intel's Software Developers' Manual located at: https://software.intel.com/sites/default/files/managed/39/c5/325462-sdm-vol-1-2abcd-3abcd.pdf. Note that this is a 4700 page manual and is an advanced guide for people who want to write low-level code (Privilege levels are described in section 6.3.5) If you are looking for ARM, you can look here: http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.dui0552a/CHDIGFCA.html Once again, the content on this page is a small part of a large book on the architecture of ARM Cortex A73 processors.

Based on the low-level code is another layer which serves as an API which the application software can call. If a program wants to read a file, it would call a function from this API. This function cannot invoke the processor to send electric signals to the storage unit or memory controller; instead it calls another method within the OS which can lay out steps to read the